Border Surveillance AI

AI-Powered Border Surveillance System
with Real-Time Anomaly Detection

PROJECT SETUP GUIDE

Step-by-Step: Clone · Configure · Own · Deploy

GTU Internship 2026 · Microsoft Elevate Program
SAL Institute of Technology and Engineering Research

Name: Jainil R. Gupta(Jay)

1. Linkedin: jainilgupta
2. Instagram: jainilgupta
3. Github: jainilgupta02

What This Guide Covers

This guide walks you through taking the **Border Surveillance AI** project and setting it up completely on your own machine with your own GitHub repository — so it is yours to submit, present, and own.

Phase 1	Prerequisites — what to install before starting
Phase 2	Clone the original project to your local machine
Phase 3	Set up Python environment and install dependencies
Phase 4	Personalise the project — make it yours
Phase 5	Create your own GitHub repository
Phase 6	Connect and push everything to your repo
Phase 7	Verify — run the pipeline and dashboard
Phase 8	Final checklist before submission

Total Setup Time: Approximately 15–25 minutes for a first-time setup. Follow every step in order and you will have a fully working project on your own GitHub.

PHASE 1 — Prerequisites

Prerequisites

Install these tools on your machine before anything else.

Required Tools

Tool	Required Version / Notes
Python 3.10+	Download from python.org — tick "Add to PATH" during install on Windows
Git	Download from git-scm.com — required to clone and push code
VS Code	Recommended editor — download from code.visualstudio.com
GitHub Account	Create a free account at github.com if you do not have one
Node.js (opt.)	Only needed if you want to run make commands — not required to run the project

Verify Your Setup

Open a terminal (Command Prompt / PowerShell / Terminal) and run these commands:

```
python --version          # Should print Python 3.10.x or higher
git --version             # Should print git version 2.x.x

# If python command is not found on Windows, try:
python3 --version
```

⚠ **Windows Users:** If Python is not recognised, open "Edit the system environment variables" → Environment Variables → add your Python install path to the PATH variable. Then restart your terminal.

PHASE 2 — Clone the Project Locally

Clone the Project Locally

Download the source code to your machine.

Step-by-Step

1 **Open a terminal and navigate to where you want the project**

Choose a folder you work in — for example your Desktop or a Projects folder.

```
# Windows (PowerShell)
cd C:\Users\YourName\Desktop

# Linux / macOS / WSL
cd ~/Desktop
```

2 **Clone the repository**

This downloads the full project code to your machine.

```
git clone https://github.com/jainilgupta02/Border-Surveillance-Project.git
```

3 **Enter the project folder**

```
cd Border-Surveillance-Project
```

4 **Confirm the files are there**

You should see src/, dashboard/, scripts/, tests/, README.md, requirements.txt etc.

```
# Windows
dir

# Linux / macOS / WSL
ls
```

☑ **Good Sign:** If you see folders like src/, dashboard/, scripts/, tests/ and files like requirements.txt and README.md — the clone worked correctly.

PHASE 3 — Set Up Python Environment

Set Up Python Environment

Create a virtual environment and install all dependencies.

1 Create a virtual environment

This keeps the project dependencies isolated from your system Python.

```
python -m venv venv

# If the above fails, try:
python3 -m venv venv
```

2 Activate the virtual environment

```
# Windows (PowerShell)
venv\Scripts\Activate.ps1

# Windows (Command Prompt)
venv\Scripts\activate.bat

# Linux / macOS / WSL
source venv/bin/activate
```

After activation, your terminal prompt will start with `(venv)` — this confirms it worked.

3 Upgrade pip

```
pip install --upgrade pip
```

4 Install all project dependencies

This will take 3–10 minutes depending on your internet speed.

```
pip install -r requirements.txt
```

⚠ **If torch fails:** Install the CPU version separately: pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu — then rerun pip install -r requirements.txt

5 Verify the installation worked

```
python -c "from ultralytics import YOLO; import cv2; print('Setup successful!')"
```

If you see `Setup successful!` — all core packages installed correctly.

6 Create your .env file

This file holds your optional cloud and email credentials. Leave fields empty to run in local-only mode.

```
# Windows
copy .env.example .env

# Linux / macOS / WSL
cp .env.example .env    # OR just create a new file called .env
```

Open .env in VS Code and fill in your values. All Azure and SendGrid fields are optional — leave them blank to run locally:

```
AZURE_STORAGE_CONNECTION_STRING=
AZURE_COSMOS_ENDPOINT=
AZURE_COSMOS_KEY=
AZURE_COSMOS_DATABASE=SurveillanceDB
AZURE_COSMOS_CONTAINER=Alerts
AZURE_STORAGE_CONTAINER_ALERTS=alert-frames
AZURE_STORAGE_CONTAINER_RESULTS=session-results
SENDGRID_API_KEY=
ALERT_FROM_EMAIL=
ALERT_TO_EMAIL=
SMTP_USER=
SMTP_APP_PASSWORD=
```

PHASE 4 — Personalise the Project

Personalise the Project

Update your name, enrollment number, and details so the project is yours.

Before pushing to your own GitHub, replace the original author details with your own. Use VS Code's **Find & Replace (Ctrl+H / Cmd+H)** to make these changes quickly across all files.

Find & Replace Reference Table

File	Find This	Replace With
README.md	Jainil Gupta (Jay Gupta)	Your Full Name
README.md	220670132018	Your Enrollment Number
README.md	@jainilgupta02	@YourGitHubUsername
README.md	Prof. Chintan Rana	Your Internal Guide Name
README.md	Adarsh Gupta	Your External Guide Name
README.md	jainilgupta02/Border-Surv...	YourUsername/Your-Repo-Name
pyproject.toml	jainilgupta02	YourGitHubUsername
pyproject.toml	Jainil Gupta	Your Full Name
pyproject.toml	jainilgupta93@gmail.com	your.email@example.com
makefile	jainilgupta02	YourGitHubUsername

Update the README Author Section

Open README.md, scroll to the Author section, and update all personal details:

```
| **Name**        | Your Full Name                          |
| **Role**        | Solo Developer                          |
| **Enrollment**  | Your Enrollment Number                  |
| **GitHub**      | [@YourUsername](https://github.com/...) |
| **Internal Guide** | Your Guide Name                      |
```

Update pyproject.toml

Open pyproject.toml and update the [tool.poetry] / [project] section with your details:

```
[project]
name = "border-surveillance-ai"
version = "1.0.0"
description = "AI-Powered Border Surveillance System with Real-Time Anomaly
Detection"
authors = [{ name = "Your Full Name", email = "your.email@example.com" }]
```

PHASE 5 — Create Your Own GitHub Repository

Create Your Own GitHub Repository

Set up a fresh empty repo on GitHub for your project.

1 Go to github.com and sign in to your account

2 Create a new repository

Click the "+" button in the top-right corner → "New repository".

3 Configure the repository settings

Repository name	Border-Surveillance-Project (or your preferred name)
Description	AI-Powered Border Surveillance System with Real-Time Anomaly Detection
Visibility	Public (required for GTU submission visibility)
Initialize	Leave unchecked — we will push existing code
Add .gitignore	None — already included in the project
Choose license	None — MIT license file already included

4 Click "Create repository"

GitHub will create an empty repo and show you the remote URL. Copy it — you will need it next.

Your remote URL will look like:

```
https://github.com/YourUsername/Border-Surveillance-Project.git
```

PHASE 6 — Connect Your Repo & Push

Connect Your Repo & Push

Disconnect from the original repo and connect to yours.

1 Remove the original remote (Jainil's repo)

```
git remote remove origin
```

2 Add your own GitHub repo as the new remote

Replace YourUsername and YourRepoName with your actual values.

```
git remote add origin https://github.com/YourUsername/Border-Surveillance-
Project.git
```

3 Verify the remote is set correctly

```
git remote -v
# Should show YOUR GitHub URL for both fetch and push
```

4 Check the current branch name

```
git branch
# Note the branch name — likely "develop" or "main"
```

5 Stage all your personalisation changes

```
git add .
git status     # Confirm only expected files are staged
```

⚠ **Before you commit:** Run git status and make sure you do NOT see: data/ folder, venv/ folder, .env file, *.pt model files (unless you want to include them). These are all in .gitignore and should not appear.

6 Commit your changes

```
git commit -m "Initial setup: personalised project for submission"
```

7 Push to your GitHub repo

```
# If your branch is "develop":
git push -u origin develop

# If your branch is "main":
git push -u origin main
```

8 Authenticate with GitHub when prompted

Enter your GitHub username. For password, use a Personal Access Token (not your GitHub password).

To create a Personal Access Token: GitHub → Settings → Developer settings → Personal access tokens → Tokens (classic) → Generate new token → select "repo" scope → copy the token.

9 Create and push the main branch (if needed)

If your working branch is "develop", also create a main branch for submission.

```
git checkout -b main
git push -u origin main
```

☑ **Success:** Go to github.com/YourUsername/Border-Surveillance-Project — you should see all your files: src/, dashboard/, scripts/, tests/, README.md with your name, and everything else.

PHASE 7 — Verify — Run the Project

Verify — Run the Project

Confirm everything works end-to-end on your machine.

Run the AI Pipeline

Make sure your virtual environment is active (you see (venv) in your prompt), then run:

```
python src/pipeline.py --video data/test_videos/dota_aerial_test.mp4 --save-frames
```

Expected output in your terminal:

```
Preprocessing complete   — frames extracted and resized to 640x640
Detection complete       — structured detections logged per frame
Anomaly scoring complete — Isolation Forest scored all frames
Alerts generated         — priority levels assigned and logged
Session saved            — data/results/session_<source>_<timestamp>.json
Alert log written        — data/alerts/alert_log.json
```

No Test Video?: If dota_aerial_test.mp4 is missing, run: python scripts/generate_test_video.py — this creates a synthetic video in data/test_videos/ for testing.

Launch the Dashboard

```
streamlit run dashboard/app.py
```

Open your browser at `http://localhost:8501`. You should see the Border Surveillance AI dashboard with alert feeds, charts, and session data.

Run the Test Suite

```
pytest tests -v

# With coverage report:
pytest tests --cov=src --cov-report=html
# Opens at: htmlcov/index.html
```

PHASE 8 — Final Checklist

Final Checklist

Tick everything before you submit your project link.

Setup Checklist

- ☐ Python 3.10+ installed and accessible from terminal
- ☐ Git installed and configured with your name/email
- ☐ Project cloned and virtual environment created
- ☐ pip install -r requirements.txt completed without errors
- ☐ .env file created (even if all fields are empty)
- ☐ Pipeline runs successfully on a test video
- ☐ Dashboard launches at http://localhost:8501
- ☐ pytest tests -v runs without failures

Personalisation Checklist

- ☐ README.md — your name, enrollment number, GitHub username updated
- ☐ README.md — your internal and external guide names updated
- ☐ pyproject.toml — your name and email updated
- ☐ makefile — GitHub username updated
- ☐ Author section in README shows your details correctly on GitHub

GitHub Checklist

- ☐ New GitHub repo created under your account
- ☐ Original remote removed, your remote added
- ☐ All code pushed — src/, dashboard/, scripts/, tests/ visible on GitHub
- ☐ README renders correctly on GitHub repo homepage
- ☐ data/ folder NOT visible on GitHub (gitignored)
- ☐ .env file NOT visible on GitHub (gitignored)
- ☐ models/ folder visible with border_yolo.pt and anomaly_model.pkl
- ☐ main branch exists and is up to date

☑ **Ready to Submit:** Once all boxes above are ticked, your project is complete, professional, and ready to share as your GitHub repository URL.

Troubleshooting

Common Errors and Fixes

Error: python not recognised

- Use python3 instead of python everywhere. On Windows, ensure Python was added to PATH during install.

Error: pip install fails with externally-managed-environment

- Create and activate your virtual environment first (step 1 of Phase 3), then retry pip install.

Error: torch install fails or times out

- Run: pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu Then retry requirements.txt.

Error: git push rejected (authentication failed)

- Use a Personal Access Token instead of your password. See Phase 6 Step 8 for how to create one.

Error: streamlit not found after pip install

- Make sure your virtual environment is activated — you should see (venv) in your prompt. Then retry.

Error: No module named cv2

- Run: pip install opencv-python-headless If on Windows try: pip install opencv-python instead.

> **Still Stuck?:** Open the project on GitHub and create an Issue with a description of your error and the full terminal output. Include your OS, Python version (python --version), and the exact command you ran.

Quick Reference — All Key Commands

```
# ── SETUP ──────────────────────────────────────────────────
python -m venv venv
source venv/bin/activate          # Linux/Mac/WSL
venv\Scripts\Activate.ps1         # Windows PowerShell
pip install --upgrade pip
pip install -r requirements.txt

# ── RUN PIPELINE ───────────────────────────────────────────
python src/pipeline.py --video data/test_videos/dota_aerial_test.mp4 --save-frames
python src/pipeline.py --camera 0                          # Live camera

# ── DASHBOARD ──────────────────────────────────────────────
streamlit run dashboard/app.py                             # →
http://localhost:8501

# ── TESTS ──────────────────────────────────────────────────
pytest tests -v
pytest tests --cov=src --cov-report=html

# ── GIT ────────────────────────────────────────────────────
git remote remove origin
git remote add origin https://github.com/YOU/YOUR-REPO.git
git add .
git commit -m "Initial personalised setup"
git push -u origin main

# ── MAKE SHORTCUTS (if make is available) ──────────────────
make setup      # create venv + install
make verify     # check all imports + models
make run        # run pipeline on test video
make dashboard  # launch streamlit
make test       # run full test suite
```

You are all set.

Your project is live, personalised, and ready to submit.

GTU Internship 2026 · Microsoft Elevate Program · SAL Institute

If you like my effort and also this guide and my project becomes helpfull for you in anyway then you can show your love:

Jay Gupta

UPI ID: jainilgupta93-2@okhdfcbank

Scan to pay with any UPI app